

Clustering Algorithms

Will J. Pearson (NCBJ)
VISTULA - 24th September 2026

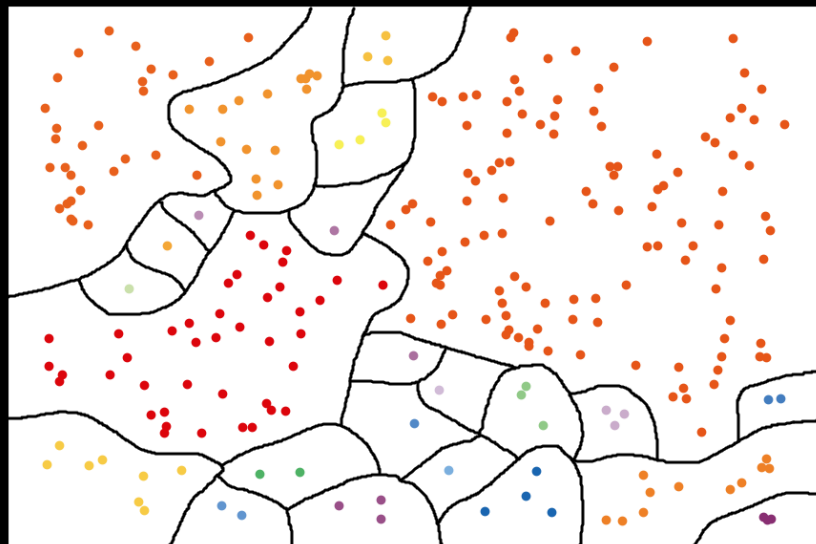
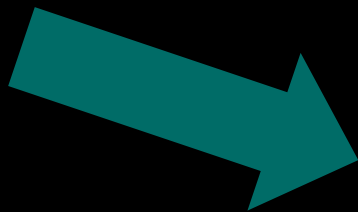
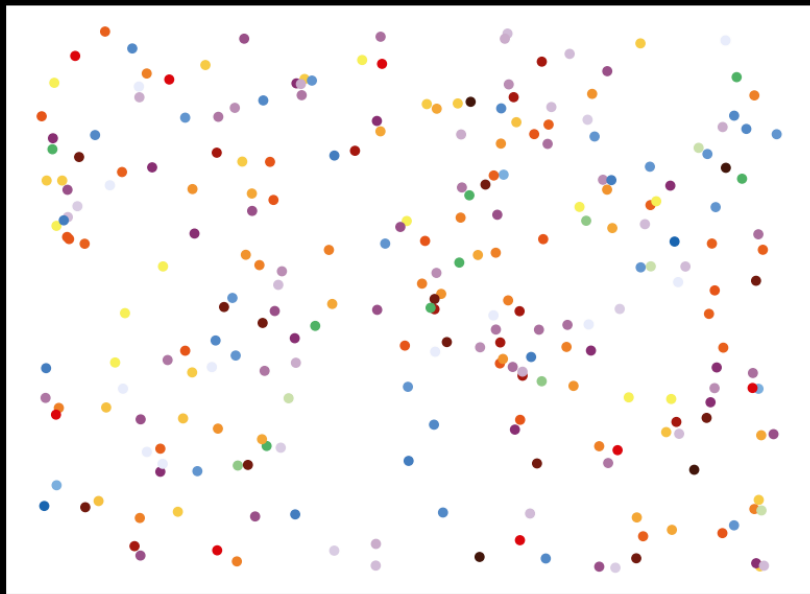


Narodowe Centrum Badań Jądrowych
National Centre for Nuclear Research



Clustering Algorithms

- We have data in d dimensions.
- These data may (or may not) group together into clusters
- These clusters are likely to contain objects with similar properties
 - e.g. blue disk galaxies, red elliptical galaxies
- To study objects in clusters, we need to find the clusters





Methods

- Minimum spanning tree (Friends of Friends)
- Density-Based Spatial Clustering of Applications with Noise
- Hierarchical Density-Based Spatial Clustering of Applications with Noise
- k-means
- Gaussian Mixture Modelling

Friends of Friends

- Calculate the distance between all points
- Define some separation between points (ϵ) for them to be considered “close”
- For each point, find the other points that are “close” (“friends”)

FoF

- For point x_i 's friends, add their friends to the same cluster
 - and then add their friends
 - and their friends
 - and their friends
 - and so on
- All points will be in a cluster
 - A point with no friends will be in a cluster on its own

Disadvantages

- FoF
 - Have to decide on what ε should be
 - i.e. what is the definition of similar/different
 - Have to compute up to n^2 separations
 - Can be reduced with careful selections
 - Can “chain” clusters together through a single point
 - As ε is fixed, struggles with variable density regions

Density-Based Spatial Clustering of Applications with Noise

- Define a radius to find neighbours (ε)
- Define a core point c_i to have ρ other points within ε , including c_i
- Point r_{d,c_i} is directly reachable from c_i if it is within ε from c_i
- Point r_{c_i} is (indirectly) reachable from c_i if it can be reached via other core points ($c_{i+1} \rightarrow c_{i+2} \rightarrow \text{etc.}$)
 - All points on the path $c_i \rightarrow r_{c_i}$ must be core point, with the possible exception of r_{c_i} itself

DBSCAN

- For a point with more than ρ points within ε (including itself), add all points within ε to its group
- If any of those new points also have ρ points within ε , add the points directly reachable to those new points to the group
- If $\rho = 2$, we recover FoF

Disadvantages

- DBSCAN
 - Have to decide ε and ρ
 - i.e. what is the definition of similar/different and how similar things should be
 - Assumes density is locally uniform (or close to it)
 - Depends on the scaling of the data
 - e.g. if you scale the dimensions, the clustering changes
 - Indirectly reachable points can be assigned different groups if the order of assignment changes

Hierarchical DBSCAN

- Extends DBSCAN to better characterise noise and “out of cluster” points
- Adds a hierarchy to the clusters based on their stability as clusters are split
- But the clustering works slightly differently to normal DBSCAN

HDBSCAN

- For point x_i , find the distance to the ρ^{th} nearest neighbour. This is the “core” distance for x_i ($core_{\rho}(x_i)$)
- We can define a distance between x_i and x_j $d(x_i, x_j)$
- We can define a “mutual reachability distance” as $d_r(x_i, x_j) = \max\{core_{\rho}(x_i), core_{\rho}(x_j), d(x_i, x_j)\}$

HDBSCAN

- $d_r(x_i, x_j)$ acts to push points in sparse regions further apart, as $core_\rho(x_i)$ or $core_\rho(x_j)$ will be larger than $d(x_i, x_j)$
- While maintaining the distance between points in dense regions
- Use these $d_r(x_i, x_j)$ to build a minimum spanning tree (with, e.g. Prim's algorithm)

HDBSCAN

- Start at an arbitrary point x_i
- Connect it to the point x_j with the smallest $d_r(x_i, x_j)$
- Of the points already in the cluster, connect the next point that is not already in the cluster, x_k , with the smallest d_r
- Repeat the above step until all points are in the cluster

HDBSCAN

- If we remove a connection from the tree, we split a cluster into two
- We can order the d_r in our tree in decreasing order
- And start removing connections, starting at the largest d_r
 - largest d_r should represent the most dis-similar points
- At some point, we need to stop removing connections

HDBSCAN

- Define how few points a cluster can have ε
 - “Clusters” that are split off as we remove connections with less than ε points are treated as noise
- Define $\lambda = 1/d_r$
- Define λ_{birth} as λ when the cluster was formed
- Define λ_{death} as λ when the cluster was split into new clusters
- Define λ_{xi} as λ when point x_i left the cluster

HDBSCAN

- With these λ we can define the stability (S) of cluster C_i as

$$S(C_i) = \sum_{x \in C_i} (\lambda_{x_i} - \lambda_{birth})$$

- The final set of clusters are the ones that maximise the total stability

$$\max \sum_{C \in \mathcal{C}} S(C)$$

- while no cluster is a subset of a different cluster

Disadvantages

- HDBSCAN
 - Have to decide ρ
 - i.e. how similar things should be
 - Slow on large datasets
 - a lot of computation to build MST and hierarchy



k -means

- Aims to divide points into k clusters
- Each point x_i belongs to the cluster that has the closest mean position
- Clusters are formed by minimising the sum of the inter-cluster variance
 - Reduce the sum of the squared Euclidean distance to the mean of each cluster

k-means

- Minimise

$$\sum_{i=1}^k \|x - \mu_i\|^2$$

where

$$\mu_i = \frac{1}{n_i} \sum_i^k x$$

and n_i is the number of points in cluster i



k-means

- Clusters found are dependent on the initial guess of where the clusters should be
 - Can change initial guess and end up with points in different clusters
- Have to choose how many clusters you want to find



k -means

- k -median
 - Like k -means but minimised the Manhattan distance to the median of the cluster
- k -medoids
 - Like k -means but the centre of a cluster is a point in that cluster that minimises the sum of distances to all other cluster members

Disadvantages

- k -means
 - Have to decide k
 - i.e. need to know how many clusters are expected
 - Sensitive to initial guess
 - Sensitive to outliers
 - k -means more than k -medians
 - Non-circular/spherical groups are not always correctly identified
 - Tries to make clusters with not too dissimilar numbers of points

Gaussian Mixture Modelling

- Assumes that the points x_i are drawn from an underlying multi-Gaussian distribution
- Similar to k -means
 - You are trying to find the mean of the cluster
- Does not require a point to belong to only one cluster
 - Can be, e.g. 20% cluster A and 80% cluster B

GMM

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{\frac{d}{2}} |\Sigma|^{\frac{1}{2}}} \exp\left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu})\right)$$

where Σ is the covariance matrix

GMM

- We want to find k clusters, so we have k Gaussian distributions
- Each distribution is then

$$p(\mathbf{x})_k = \frac{1}{(2\pi)^{\frac{d}{2}} |\boldsymbol{\Sigma}_k|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T \boldsymbol{\Sigma}_k^{-1}(\mathbf{x} - \boldsymbol{\mu}_k)\right)$$

- Each distribution has a weight π such that

$$\sum_k \pi_k = 1$$

GMM

- The probability at any point is then

$$P(\mathbf{x}_i) = \sum_k \pi_k \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \Sigma_k)$$

- The probability that Gaussian k formed point x_i
 - Also called responsibility

$$\gamma_{ik} = P(k | \mathbf{x}_i) = \frac{\pi_k \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \Sigma_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_j, \Sigma_j)}$$

GMM

- Use Expectation-Maximisation (EM) to find the parameters of the Gaussians
 - Expectation (E)
 - calculate γ_{ik}
 - and log likelihood
- $$L(\boldsymbol{\mu}, \boldsymbol{\Sigma}, \pi) = \prod_{i=1}^n \sum_{k=1}^K \gamma_{ik} \log(\pi_k \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k))$$
- Maximisation (M)

GMM

- At each M step, update μ , Σ , and π

$$\mu_k = \frac{\sum_{i=1}^n \gamma_{ik} \mathbf{x}_i}{\sum_{i=1}^n \gamma_{ik}}$$

$$\Sigma_k = \frac{\sum_{i=1}^n \gamma_{ik} (\mathbf{x}_i - \mu_k) (\mathbf{x}_i - \mu_k)^T}{\sum_{i=1}^n \gamma_{ik}}$$

$$\pi_k = \frac{1}{n} \sum_{i=1}^n \gamma_{ik}$$



GMM

- Repeat the E- and M-steps until it converges
- Like k -means, you have to choose k and the initial parameters of the Gaussians

Disadvantages

- GMM
 - Have to decide k
 - i.e. need to know how many clusters are expected
 - Assumes the underlying distribution can be modelled by a (smallish) series of Gaussians
 - Sensitive to outliers
 - High number of parameters makes it computationally expensive

Applications

- Have some data, don't know what they are
- Clusters in low dimensional space should have similar properties in high dimensional space
- Can study a few objects in a cluster to work out what the rest are
 - Can use known classifications in a group to find new objects of the same type
- Can find outliers
 - These can often be interesting
 - Or just some technical error